

- JS2-数据类型
 - 3 数据类型转换
 - 3.1 隐式类型转换
 - 3.1.1 数学运算符中的类型转换
 - 1. 减、乘、除
 - 2. 加
 - 3.1.2 逻辑语句中的类型转换
 - 1. 单个变量
 - 2. == 比较
 - 附录：

JS2-数据类型

3 数据类型转换

JavaScript 是一种 **弱类型** 和 **动态类型** 的编程语言，这意味着变量的类型可以在运行时动态改变，而无需在声明时指定类型。这种特性通过动态类型、隐式类型转换和灵活的类型检测等机制实现。

3.1 隐式类型转换

3.1.1 数学运算符中的类型转换

因为 JS 并没有类型声明，所以任意两个变量或字面量，都可以做加减乘除。

1. 减、乘、除

在对各种非 **Number** 类型运用数学运算符(- * /)时，会先将非 **Number** 类型转换为 **Number** 类型。

```
console.log(1 - true); // 0, 首先把 true 转换为数字 1, 然后执行 1 - 1
console.log(1 - null); // 1, 首先把 null 转换为数字 0, 然后执行 1 - 0
console.log(1 * undefined); // NaN, undefined 转换为数字是 NaN
console.log(2 * ['5']); // 10, ['5'] 首先会变成 '5', 然后再变成数字 5, 然后执行 2 * 5
```

```
// NaN
console.log(Number(undefined)); // undefined->NaN
console.log(Number("123abc")); // 存在非数字: string->NaN
console.log(Number(function({}))); // function->NaN
console.log(Number({})); // Object->NaN
console.log(Number(['123','abc'])); // 存在非数字: array->NaN
// 0
console.log(Number([])); // [] -> 0
console.log(Number(false)); // false -> 0
console.log(Number('')); // 空字符串'' -> 0
console.log(Number(null)); // null -> 0
// Error
console.log(Number(Symbol(1))); // TypeError: Cannot convert a Symbol value to a
number
console.log(Number(10n)); // TypeError: Cannot mix BigInt and other types,
use explicit conversions
```

2. 加

由于JS中 **+** 还可以用来拼接字符串，所以必须特殊对待。

优先级从高到低有如下规则：

- 当一侧存在 **String** 类型，**+** 被识别为字符串拼接，优先将另一侧转换为字符串类型。
- **Number** 类型 + 原始类型 或 原始类型 + 原始类型，将原始类型转换为 **Number** 类型。
- **Number** 类型 + 引用类型 或 引用类型 + 引用类型，将引用类型、**Number** 类型转换成字符串后拼接。

```
// 规则1
console.log(1 + '1'); // '11'
// 规则2
console.log(true + false); // 1
console.log(undefined + 1); // NaN
console.log(null + 1); // 1
// 规则3
console.log(123 + {}); // '123[object Object]'
console.log([,,,] + [,,]); // ',,'
// [,,,].toString() 等于 ",,"
// [,,].toString() 等于 ",,"
// [,,,] + [,,] = ",," + ",," = ",,,"
```

```
console.log(String({})); // '[object Object]'
console.log(String([])); // ''
console.log(String([undefined])); // ''
```

```
console.log(String(function({}))); // 'function(){}'  
console.log(String(null)); // 'null'
```

3.1.2 逻辑语句中的类型转换

当我们使用 `if while` 语句时，我们期望表达式是一个 `Boolean`，所以一定伴随着隐式类型转换。

1. 单个变量

如果只有单个变量，会先将变量转换为 `Boolean` 值。

```
// 只有以下类型转变为Boolean后的值为false  
console.log(Boolean(null));  
console.log(Boolean(undefined));  
console.log(Boolean(''));  
console.log(Boolean(NaN));  
console.log(Boolean(0));  
console.log(Boolean(false));  
// 易错  
console.log(Boolean([])); // true  
console.log(Boolean([undefined])); // true  
console.log(Boolean({})); // true
```

2. == 比较

规则：

- `NaN`和其他任何类型比较永远返回 `false`（包括和他自己）

```
console.log(NaN == NaN); // false
```

- `Boolean`和其他任何类型比较，`Boolean` 首先被转换为 `Number` 类型。
- `String`和 `Number`比较，先将 `String`转换为 `Number`类型。
- `null == undefined`比较结果是 `true`，除此之外，`null`、`undefined`和其他任何结果的比较值都为 `false`。

```
// true  
console.log(null == undefined);  
console.log(null == null);  
console.log(undefined == undefined);
```

- 原始类型和引用类型做比较时，引用类型会依照 **ToPrimitive** 规则转换为原始类型。**ToPrimitive** 规则，是引用类型向原始类型转变的规则，它遵循先 **valueOf** 后 **toString** 的模式期望得到一个原始类型。如果还是没法得到一个原始类型，就会抛出 **TypeError**。

```
console.log([] == ![]); // true
// 1. ![] -> false
// 2. 规则2, false -> 0
// 3. 规则5, [] -> []的valueOf值为0
// 4. 此时等式变为 0 == 0 true
```

```
console.log([undefined] == false); // true
// 规则2, false -> 0
// 规则5, [undefined].toString() -> ''
// 规则3, '' -> 0
// 0 == 0 true
```

附录：

类型	值	to Boolean	to Number	to String
Boolean	true	true	1	"true"
Boolean	false	false	0	"false"
Number	123	true	123	"123"
Number	Infinity	true	Infinity	"Infinity"
Number	0	false	0	"0"
Number	NaN	false	NaN	"NaN"
String	""	false	0	""
String	"123"	true	123	"123"
String	"123abc"	true	NaN	"123abc"
String	"abc"	true	NaN	"abc"
Null	null	false	0	"null"
Undefined	undefined	false	NaN	"undefined"
Function	function(){}	true	NaN	"function(){} "
Object	{}	true	NaN	"[object Object]"
Array	[]	true	0	""
Array	["abc"]	true	NaN	"abc"
Array	["123"]	true	123	"123"
Array	["123", "abc"]	true	NaN	"123,abc"